United States Patent Application Publication	20250284606
Kind Code	A1
Publication Date	September 11, 2025
Inventor(s)	Bhatia; Shrey et al.

PROCESSOR WITH DEBUG PIPELINE

Abstract

An example system includes execution circuitry disposed within a first power domain; a sub-system to selectively operate to perform operations on the system and that is disposed within a second power domain that is separate from the first power domain; power control circuitry coupled to the sub-system; and detection circuitry coupled to the power control circuitry. The power control circuitry is configured to cause power to be supplied to the sub-system when the detection circuitry detects that the external component is coupled to the system, and configured to disable power supply to the sub-system when the detection circuitry detects that the external component is not coupled to the system.

Inventors: **Bhatia; Shrey (Freising, DE), Wiencke; Christian (Garching, DE), Stingl; Armin (Erding, DE), Ledwa; Ralph (Buching, DE), Lutsch; Wolfgang (Freising, DE)**

Applicant: **TEXAS INSTRUMENTS INCORPORATED** (Dallas, TX)

Family ID: **54322128**

Appl. No.: **19/220427**

Filed: **May 28, 2025**

Related U.S. Application Data

parent US continuation 18474690 20230926 parent-grant-document US 12353308 child US 19220427
parent US continuation 18175607 20230228 parent-grant-document US 11803455 child US 18474690
parent US continuation 17146584 20210112 parent-grant-document US 11593241 child US 18175607
parent US continuation 16102193 20180813 parent-grant-document US 10891207 child US 17146584
parent US continuation 15200900 20160701 parent-grant-document US 10049025 child US 16102193

parent US continuation 14255055 20140417 parent-grant-document US 9384109 child US 15200900

Publication Classification

Int. Cl.: **G06F11/273** (20060101); **G06F11/26** (20060101); **G06F11/30** (20060101); **G06F11/34** (20060101); **G06F11/362** (20250101); **G06F11/3698** (20250101)

U.S. Cl.:

CPC **G06F11/273** (20130101); **G06F11/26** (20130101); **G06F11/3024** (20130101); **G06F11/3089** (20130101); **G06F11/348** (20130101); **G06F11/3636** (20130101); **G06F11/3648** (20130101); **G06F11/3656** (20130101); **G06F11/3698** (20250101);

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS [0001] This U.S. patent application is a continuation of U.S. patent application Ser. No. 18/474,690, filed Sep. 26, 2023, which is a continuation of U.S. patent application Ser. No. 18/175,607, filed Feb. 28, 2023, now U.S. Pat. No. 11,803,455, which is a continuation of U.S. patent application Ser. No. 17/146,584, filed Jan. 12, 2021, now U.S. Pat. No. 11,593,241, which is a continuation of U.S. patent application Ser. No. 16/102,193, filed Aug. 13, 2018, now U.S. Pat. No. 10,891,207, which is a continuation of U.S. patent application Ser. No. 15/200,900, filed Jul. 1, 2016, now U.S. Pat. No. 10,049,025, which is a continuation of U.S. patent application Ser. No. 14/255,055, filed Apr. 17, 2014, now U.S. Pat. No. 9,384,109, each of which is hereby incorporated by reference in its entirety.

BACKGROUND
[0002] Embedded software development and debugging using processors, such as microcontrollers, can be challenging. Accordingly, many processors include on-chip debug features to facilitate the software development process. Among the most fundamental debug features are program breakpoints. Program breakpoints allow the processor to be halted when software execution reaches a specific address. Once the processor is halted, a debug tool can examine the state of system memory and registers in the processor by issuing instructions to be executed on the processor through a debug protocol. Once the examination is completed, the debug tool can return the CPU to normal mode, and execution will continue until the next breakpoint. Some processors include circuitry to facilitate use of breakpoints. Such “hardware breakpoints” are non-intrusive, and have virtually no impact on software execution until the breakpoint triggers.
SUMMARY
[0003] In an example, a system comprises execution circuitry disposed within a first power domain; a sub-system disposed within a second power domain that is separate from the first power domain, in which the sub-system is configured to selectively operate to perform operations on the system; power control circuitry coupled to the sub-system; and detection circuitry coupled to the power control circuitry. The power control circuitry is configured to cause power to be supplied to the sub-system when the detection circuitry detects that the external component is coupled to the system, and disable power supply to the sub-system when the detection circuitry detects that the external component is not coupled to the system.
[0004] In another example, a system comprises a processor having a first power domain and a second power domain that is separate from the first power domain, in which the processor includes a first pipeline and first control logic within the first power domain, and a second pipeline and

second control logic within the second power domain. The system further comprises power circuitry coupled to the second power domain; and detection circuitry coupled to the power circuitry. In response to the detection circuitry detecting coupling of an external component to the processor, the power circuitry is configured to cause power to be supplied to the second power domain, and in response to power being supplied to the second power domain, the second control logic is configured to exchange signals with the first control logic to cause processing information to advance through the second pipeline in synchronization with advancement of an instruction through the first pipeline.

[0005] In still another example, a method comprises powering a first power domain of a system; receiving an instruction via a bus; processing the instruction using a first pipeline in the first power domain; determining whether an external component is coupled to the system; causing power to be provided to a processing circuit in a second power domain of the system to initiate servicing of an event in the system by the processing circuit when it is determined that the external component is coupled to the system; and based on power being provided to the processing circuit, exchanging signals between first logic in the first power domain and second logic in the second power domain to cause information associated with the servicing of the event to advance through a second pipeline in the second power domain in synchronization with advancement of the instruction through the first pipeline.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] For a detailed description of exemplary embodiments of the invention, reference will now be made to the accompanying drawings in which:

[0007] FIG. **1** shows a block diagram of a processor in accordance with various embodiments;

[0008] FIG. **2** shows a debug system in a processor in accordance with various embodiments; and

[0009] FIG. **3** shows a flow diagram for a method for debugging in accordance with various embodiments.

NOTATION AND NOMENCLATURE

[0010] Certain terms are used throughout the following description and claims to refer to particular system components. As one skilled in the art will appreciate, companies may refer to a component by different names. This document does not intend to distinguish between components that differ in name but not function. In the following discussion and in the claims, the terms “including” and “comprising” are used in an open-ended fashion, and thus should be interpreted to mean “including, but not limited to . . . ” Also, the term “couple” or “couples” is intended to mean either an indirect or direct electrical connection. Thus, if a first device couples to a second device, that connection may be through a direct electrical connection, or through an indirect electrical connection via other devices and connections. The recitation “based on” is intended to mean “based at least in part on.” Therefore, if X is based on Y, X may be based on Y and any number of additional factors.

DETAILED DESCRIPTION

[0011] The following discussion is directed to various embodiments of the invention. Although one or more of these embodiments may be preferred, the embodiments disclosed should not be interpreted, or otherwise used, as limiting the scope of the disclosure, including the claims. In addition, one skilled in the art will understand that the following description has broad application, and the discussion of any embodiment is meant only to be exemplary of that embodiment, and not intended to intimate that the scope of the disclosure, including the claims, is limited to that embodiment.

[0012] Processor on-chip debugging systems that include hardware breakpoints, watchpoints and

the like require that the processor include circuitry dedicated to the breakpoints and other debugging functions. In conventional processor on-chip debugging circuitry, the processor execution pipeline includes circuitry that allows instruction breakpoint information to propagate through the pipeline with the instruction until the instruction reaches a specified pipeline stage and the breakpoint information is applied to halt instruction execution. Because this additional debug circuitry is integrated into the processor's execution pipeline, the circuitry consumes energy whenever the processor is executing instructions, regardless of whether software is being debugged, thereby reducing processor power efficiency.

[0013] Embodiments of the present disclosure provide on-chip debugging that includes hardware breakpoints and the like with reduced energy consumption. The processor disclosed herein provides a debug pipeline that is separate from the processor's execution pipeline. Breakpoint information for an instruction propagates through the debug pipeline in lock-step with the propagation of the instruction through the execution pipeline. Because the debug pipeline is separate from the execution pipeline, the debug pipeline and other processor on-chip debugging features need only be powered during a debugging session. Accordingly, embodiments of the processor include separate and independent execution and debug power domains. Components located in the debug power domain, such as the debug pipeline, debug address comparison logic, etc. are powered only during debugging, which reduces overall processor energy consumption in the vast majority of execution conditions, because software development time, during which debugging features are used, constitutes only a tiny fraction of overall processor operation time.

[0014] FIG. **1** shows a block diagram of a processor **100** in accordance with various embodiments. The processor **100** may be a general purpose microprocessor, a digital signal processor, a microcontroller, or other computing device that executes instructions retrieved from an instruction memory. The processor **100** includes execution logic **102** and a debug system **104**. The execution logic **102** includes circuitry that executes instructions retrieved from memory. The debug system **104** is coupled to the execution logic **102**. The debug system **104** includes logic that controls execution of instructions in the execution logic **102** to allow for debugging of instructions being executed by the execution logic **102**. For example, the debug system **104** may halt execution of instructions by the execution logic **102** if the execution logic **102** executes an instruction retrieved from a particular memory address. Signals **110** exchanged by the execution logic **102** and the debug system **104** are applied to synchronize the operation of the execution logic and the debug system **104**.

[0015] The processor **100** includes an execution power domain **106** and a debug power domain **108**. The execution logic **102** is disposed in the execution power domain **106**. The debug system **104** is disposed in the debug power domain **108**. The debug power domain **108** is separate and distinct from the execution power domain **106**. The execution power domain **106** is an area of the processor **100** that provides power to the execution logic **102** to allow the execution of instructions. Accordingly, the execution power domain **106** provides power whenever instructions are to be executed (e.g., when power is provided to the processor **100**). The debug power domain **108** is an area of the processor **100** that provides power to the debug system **104** to allow debugging of an instruction stream being executed by the execution logic **102**. The debug power domain **108** provides power only while the debug system **104** is being used to debug an instruction stream executed by the execution logic **102**. Because the debug power domain **108** provides power to the debug system **104** only while debugging, the processor **100** uses less energy than a processor that includes debugging circuitry that is powered whenever the processor and/or the execution logic is powered. The processor **100** provides the debugging capabilities of a conventional processor while consuming less energy than a conventional processor because the debug system **104** is only powered while debugging.

[0016] FIG. **2** shows the processor debug system **104** and other subsystems of the processor **100** in greater detail. The execution logic **102** includes an execution pipeline **202** and execution pipeline

control logic **210**. The debug system **104** includes a debug pipeline **214**, debug pipeline control logic **212**, comparators **222** and registers **224**. The processor **100** also includes debug tool detection circuitry **228** and debug power circuitry **230**.

[0017] The execution pipeline **202** includes a plurality of execution stages **204**, **206**, **208** that incrementally execute an instruction. For example, the execution pipeline **202** may include a fetch stage **204**, a decode stage **206**, and an execute stage **208**. Other embodiments may include different execution stages and/or a different number of execution stages.

[0018] The fetch stage **204** retrieves instructions from instruction memory for execution by the processor **100**. The instruction memory is a storage device, such as a random access memory (volatile or non-volatile) that stores instructions to be executed. The instruction memory may an internal component of the processor **100**, or alternatively may be external to the processor **100**. The fetch stage **204** provides the retrieved instructions to the decode stage **206**.

[0019] The decode stage **206** examines the instructions received from the fetch stage **204**, and translates each instruction into controls suitable for operating the execute stage **208**, processor registers, and other components of the processor **100** to perform operations that effectuate the instructions. The decode stage **206** provides control signals to the execute stage **208** that cause the execute stage **208** to carry out the operations needed to execute each instruction.

[0020] The execute stage **208** includes arithmetic circuitry, shifters, multipliers, registers, logical operation circuitry, etc. that are arranged to manipulate data values as specified by the control signals generated by the decode stage **206**. Some embodiments of the processor **100** may include multiple execution pipelines **202** that provide the same or different data manipulation capabilities.

[0021] The execution control logic **210** controls the propagation of instructions through the execution stages of the execution pipeline **202**. For example, if execution of a given instruction in the execute stage **208** requires a particular number of clock cycles, the execution control logic **210** may advance instructions in the execution pipeline **202** on expiration of the particular number of clock cycles.

[0022] The debug pipeline **214** includes a plurality of debug stages **216**, **218**, **220**. The debug stages propagate debug information through the debug pipeline **214**. The debug pipeline **214** has the same number of debug stages as the execution pipeline **202** has execution stages. Debug information in the debug pipeline **214** advances through the debug pipeline **214** in synchronization with the advancement of an instruction corresponding to the debug information through the execution pipeline **202**.

[0023] The debug pipeline control logic **212** controls the propagation of the debug information through the stages **216**-**220** of the debug pipeline **214**. The debug pipeline control logic **212** is coupled to the execution pipeline control logic **210**. Signals **110** provided to the debug pipeline control logic **212** by the execution pipeline control logic **210** indicate when an instruction is propagating from stage to stage of the execution pipeline **212**. Responsive to the signals **110**, the debug pipeline control logic **212** causes debug information to advance through the debug pipeline **214** in lock-step with advancement of the instruction corresponding to the debug information through the execution pipeline **202**. For example, when an instruction is retrieved into the fetch stage **204**, debug information corresponding to the instruction is loaded into the debug stage **216**. When the instruction advances from the fetch stage **204** to the decode stage **206**, the debug information advances from debug stage **216** to debug stage **218**. Similarly, when the instruction advances from the decode stage **206** to the execute stage **208**, the debug information advances from debug stage **218** to debug stage **220**. Advancement through the debug pipeline stages **216**-**220** is synchronized with advancement of the instruction through the execution pipeline stages **204**-**208** by the signals **110** provided to the debug pipeline control logic **212** by the execution pipeline control logic **210**.

[0024] The registers **224** store memory addresses of instructions and/or data being used in debugging. For example, an instruction address stored in a selected register **224** may be a

breakpoint address used to halt instruction execution in the execution pipeline **202** when an instruction at the address in the selected register **224** is executed, decoded, fetched, etc. Similarly, an address stored in a selected register **224** may be a watchpoint address used to halt instruction execution in the execution pipeline **202** when the address in the selected register **224** is accessed for reading or writing.

[0025] The registers **224** are coupled to the comparators **222**. The comparators **222** compare the address values stored in the registers **224** to the instruction addresses driven onto the instruction bus **232** by the fetch stage **204**. The instruction bus **232** conveys instructions and instruction addresses between instruction memory and the fetch stage **204**. The output of comparators **222** is included in the debug information loaded into the debug pipeline **214** and propagated through the stages **216-220**. When debug information indicating a positive address comparison propagates into a selected stage of the debug pipeline **214**, the debug pipeline control logic **212** notifies the execution pipeline control logic **210**, via signals **110**, and the execution pipeline control logic **210** can halt execution of instructions in the execution pipeline **202**.

[0026] The registers **236** store values of data used in debugging. For example, a data value stored in a selected register **236** may be used to halt instruction execution in the execution pipeline **202** when the data value is output by the execute stage **208**.

[0027] The registers **236** are coupled to the comparators **234**. The comparators **234** compare the data values stored in the registers **236** to the data values driven onto the data bus by the execution pipeline **202**. The output of comparators **234** may be provided to a debug pipeline stage **216-220** corresponding to the execution pipeline stage **204-208** that performed the transaction associated with the data value. For example, if the execute stage **208** drives a data value onto the data bus for storage in a register of the processor **100**, the output of the comparators **234** may be provided to debug pipeline stage **220**.

[0028] Some embodiments of the processor **100** may include multiple instances of the execution pipeline **202**, or a portion thereof. Such embodiments may include an instance of the debug pipeline **214** coupled to each instance of the execution pipeline **202** to provide debugging services with regard to instructions executing in the execution pipeline **202**.

[0029] As explained above, the debug system **104** is powered only while software is being debugged. In some embodiments, the processor **100** detects whether a debug tool is connected to the processor **100**, and controls provision of power to the debug system **104** based on whether a debug tool is connected to the processor **100**. The debug tool detection circuitry **228** detects whether a debug tool is connected to the processor **100**. The debug tool detection circuitry **228** may detect connection of a debug tool to the processor **100** by identifying a clock signal generated by the debug tool and provided to the processor **100**. In other embodiments, the debug tool detection circuitry **228** may detect connection of a debug tool by a different method. The debug tool detection circuitry **228** provides a signal indicating whether a debug tool is connected to the processor **100** to the debug power circuitry **230**. The debug power circuitry **230** switches power to the debug system **104** (i.e., the debug power domain **108**) when the debug tool detection circuitry **228** indicates that a debug tool is connected to the processor **100**, and disconnects power from the debug power system **104** when the debug tool detection circuitry **228** indicates that the debug tool is not connected to the processor **100**. An execution power system powers the execution logic **102** independently of the debug system **104**.

[0030] FIG. **3** shows a flow diagram for a method **300** for debugging in a processor in accordance with various embodiments. Though depicted sequentially as a matter of convenience, at least some of the actions shown can be performed in a different order and/or performed in parallel. Additionally, some embodiments may perform only some of the actions shown.

[0031] In block **302**, the processor **100** is powered, the execution logic **102** is powered, and the debug system **104** is not powered because a debug tool is not connected to the processor **100**. The debug tool detection circuitry **228** is monitoring a debug tool port of the processor **100** to determine

whether a debug tool is connected to the processor **100**.
[0032] In block **304**, the debug tool detection circuitry **228** detects connection of a debug tool to the processor **100**, and in response to detection of connection of the debug tool, power is switched to the debug system **104** and the debug system **104** is enabled to provide debugging functionality.
[0033] In block **306**, the processor **100** is executing instructions. As an instruction is loaded into the execution pipeline **202** to be executed in block **306**, debug information corresponding to the instruction is loaded into the debug pipeline **214** in block **308**. The debug information may indicate whether an address of the instruction or of data accessed by the instruction corresponds to an address stored in the registers **224**.
[0034] In block **310**, the debug information is propagated through the debug pipeline **214** in synchronization with propagation of the corresponding instruction through the execution pipeline **202**.
[0035] In block **312**, the processor **100** applies the debug information in the debug pipeline **214** to control instruction execution in the execution pipeline **202**. For example, propagation of the debug information to a given stage of the debug pipeline **214** may halt execution of instructions in the execution pipeline **202**.
[0036] In block **314**, the debug tool detection circuitry **228** is monitoring the debug tool port of the processor **100**, and determines that the debug tool has been disconnected from the processor **100**.
[0037] In block **316**, responsive to detection of the disconnection of the debug tool, power is disconnected from the debug system **104**. Power to the execution logic **102** is independent of power to the debug system **104**. Accordingly, when power is disconnected from the debug system **104**, the execution logic **102** remains powered and may execute instructions while the debug system **104** is unpowered.
[0038] The above discussion is meant to be illustrative of the principles and various embodiments of the present invention. Numerous variations and modifications will become apparent to those skilled in the art once the above disclosure is fully appreciated. It is intended that the following claims be interpreted to embrace all such variations and modifications.

Claims

1. A system comprising: execution circuitry disposed within a first power domain; a sub-system configured to selectively operate to perform operations on the system, the sub-system disposed within a second power domain that is separate from the first power domain; power control circuitry coupled to the sub-system; and detection circuitry coupled to the power control circuitry, wherein the power control circuitry is configured to: cause power to be supplied to the sub-system when the detection circuitry detects that the external component is coupled to the system, and disable power supply to the sub-system when the detection circuitry detects that the external component is not coupled to the system.
2. The system of claim 1, wherein the execution circuitry includes an execution pipeline that is configured to process an instruction, and wherein the sub-system includes: a comparator configured to determine that the instruction is associated with a serviceable event in the system; and control logic configured to: determine whether the serviceable event is associated with the execution pipeline; and trigger service of the serviceable event, when it is determined that the serviceable event is associated with the execution pipeline, upon processing the instruction by the execution pipeline.
3. The system of claim 2, wherein the detection circuitry is configured to detect coupling of the external component to the system based on a clock signal received from the external component.
4. The system of claim 2, wherein: the execution pipeline includes a set of execution stages; and the sub-system includes a processing pipeline including a set of processing stages.
5. The system of claim 4, wherein the control logic is configured to cause an indication of the

serviceable event to propagate through the set of processing stages as the instruction propagates through the set of execution stages.

6. The system of claim 2, wherein: the sub-system includes a register configured to store a first address; and the comparator is configured to determine that the serviceable event is associated with instruction by comparing the first address to a second address associated with the instruction.

7. The system of claim 4, wherein the execution pipeline is configured to halt execution of the instruction based on the serviceable event being triggered.

8. The system of claim 4, wherein the set of execution stages includes a fetch stage, a decode stage, and an execute stage.

9. A system comprising: a processor having a first power domain and a second power domain that is separate from the first power domain, the processor including a first pipeline and first control logic within the first power domain, and a second pipeline and second control logic within the second power domain; power circuitry coupled to the second power domain; and detection circuitry coupled to the power circuitry; wherein, in response to the detection circuitry detecting coupling of an external component to the processor, the power circuitry is configured to cause power to be supplied to the second power domain, and wherein in response to power being supplied to the second power domain, the second control logic is configured to exchange signals with the first control logic to cause processing information to advance through the second pipeline in synchronization with advancement of an instruction through the first pipeline.

10. The system of claim 9, wherein the processor is configured to use the processing information to control execution of the instruction in the first pipeline.

11. The system of claim 9, wherein the power circuitry is configured to disable supply of power to the second power domain when coupling of the external component to the processor is not detected by the detection circuitry.

12. The system of claim 9, wherein the processor further comprises: a register in the second power domain and configured to store addresses; and a comparator in the second power domain and configured to compare addresses in the instruction, which is input to the first pipeline, to the addresses stored in the register, and output a result that is included in the processing information.

13. The system of claim 9, wherein the processor further comprises: a register in the second power domain and configured to store values for use in performing processing; and a comparator in the second power domain and configured to compare data values in the instruction, as it is output from the first pipeline, to the values stored in the register, and output a result to the second control logic.

14. The system of claim 13, wherein, when the comparator determines that a data value, among the data values in the instruction output from the first pipeline, matches a value, among the values stored in the register, the second control logic notifies the first control logic.

15. A method comprising: powering a first power domain of a system; receiving an instruction via a bus; processing the instruction using a first pipeline in the first power domain; determining whether an external component is coupled to the system; causing power to be provided to a processing circuit in a second power domain of the system to initiate servicing of an event in the system by the processing circuit when it is determined that the external component is coupled to the system; and based on power being provided to the processing circuit, exchanging signals between first logic in the first power domain and second logic in the second power domain to cause information associated with the servicing of the event to advance through a second pipeline in the second power domain in synchronization with advancement of the instruction through the first pipeline.

16. The method of claim 15, further comprising: using the information associated with the servicing of the event to control execution of the instruction in the first pipeline.

17. The method of claim 15, further comprising: disabling supply of power to the second power domain when coupling of the external component to the system is not detected.

18. The method of claim 16, wherein the using of the information associated with the servicing of the event to control execution of the instruction in the first pipeline includes halting execution of

the instruction in the first pipeline.

19. The method of claim 18, wherein the using of the information associated with the servicing of the event to control execution of the instruction in the first pipeline includes restarting execution of the instruction in the first power domain based on signals exchanged between the first logic and the second logic.

20. The method of claim 15, wherein the event includes a debug event, and the information includes debugging information.